Chapter 6 "Thread Programming"

according to the book
"Parallel Programming"
by T. Rauber and G. Rünger
Springer 2010

5. March 2013

Section outline

1. Pthread Programming

2. OpenMP

3. Java Threads

4. Java Concurrent Packaget

Creation and Management of Threads

- When a Pthreads program is started, a single **main thread** is active, executing the `main()` function of the program.

- **Creation** of a thread:

```
int pthread_create(pthread_t *thread,
                   pthread_attr_t *attr,
                   void *(*start_routine)(void*),
                   void *arg);
```

  - `thread` – **Thread-ID** for identification of the thread created;
  - `attr` – **Attribute object** for setting the properties of the thread; `NULL` creates a thread with **default properties**;
  - `start_routine` – start function of the new thread.
  - `arg` – **Parameter** for the start function.

- **note**: only **one parameter** can be passed to the start function;
  → **multiple parameters** have to be **packed** into a single **structure**.

Creation and Management of Threads

- **Termination** of a thread:

```
int pthread_exit(void *valuep);
```

`valuep` – **return value** of the terminated thread to the calling thread;
A thread is terminated **implicitly** if it **leaves its start function**.

- **Waiting** for the **termination** of a different thread:

```
int pthread_join(pthread_t thread, void **valuep);
```

  - `thread` – TID of the thread, whose termination should be waited for.
  - `valuep` – pointer to the data field, which will be filled with the return value.

Effect: The **calling thread** is **blocked** until the specified thread is **terminated**.
Only **one thread** can wait for the termination of another **one**. If `pthread_join()` is called by multiple threads, only one of those calls will be successful with the rest returning the error code `ESRCH`.

Example: Passing multiple parameters to a thread function

```
#include <pthread.h>
#include <stdio.h>
typedef struct _thrarg_t {
    int rows, cols;
} thrarg_t;
void *loopthread(void *arg) {
    int j, k;
    thrarg_t *thrarg = (thrarg_t*)arg;
    for (j = 0; j < thrarg->rows; j++)
        for (k = 0; k < thrarg->cols; k++)
            printf("loop iteration %d \t %d\n", j, k);
}
int main(int argc, char** argv) {
    pthread_t tid;
    thrarg_t thrarg;
    thrarg.rows = 10; thrarg.cols = 20;
    pthread_create(&tid, NULL, loopthread, &thrarg);
    pthread_join(tid, NULL); // necessary, since thrarg
                             // is a local variable
}
```

Example: Race conditions

If the same parameters are passed to **multiple threads**, **race conditions** might arise:

```
int main(int argc, char** argv) {
    pthread_t tid1, tid2;

    thrarg_t thrarg;

    thrarg.rows = 10; thrarg.cols = 20;
    pthread_create(&tid1, NULL, loopthread, &thrarg);
    thrarg.rows = 20; thrarg.cols = 30;
    pthread_create(&tid2, NULL, loopthread, &thrarg);

    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
}
```

Depending on the execution time of the created threads **errors** might occur, if `thrarg` is overwritten by the main thread **before** `tid1` is executed.

Example: Fixed-size matrix multiplication (1)

A **separate thread** is used for each element of the target matrix.

```
#include <pthread.h>
#define SIZE 8

typedef struct matrix_type_t {
    int row, col;
    double (*MA)[SIZE], (*MB)[SIZE], (*MC)[SIZE];
  } matrix_type_t;

void *thread_mult(void *arg) {
  matrix_type_t *work = (matrix_type_t*)arg;
  int i, r = work->row, c = work->col;

  work->MC[r][c] = 0.0;
  for (i = 0; i < SIZE; i++)
    work->MC[r][c] += work->MA[r][i] * work->MB[i][c];
  return NULL;
}
```

Example: Fixed-size matrix multiplication (2)

```
 double MA[SIZE][SIZE], MB[SIZE][SIZE], MC[SIZE][SIZE];
 matrix_type_t *work;
 pthread_t thread[SIZE][SIZE];


int main(int argc, char* argv[]) {
 int row, col;
for (row = 0; row < SIZE; row++)
   for (col = 0; col < SIZE; col++) {
     work = (matrix_type_t*)malloc(sizeof(matrix_type_t));
     work->row = row; work->col = col;
     work->MA = MA; work->MB = MB; work->MC = MC;
     pthread_create(&thread[row][col], NULL,
                    thread_mult, work);
   }
 for (row = 0; row < SIZE; row++)
   for (col = 0; col < SIZE; col++)
       pthread_join(thread[row][col]);
}
```

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Thread Coordination – Motivation

- **Threads of a process** share a **common address space** and can be interrupted by the scheduler at **any point** during their execution;
  → **race conditions** can occur;
  → avoidance by the use of **coordination mechanisms**;

- **Example**: Computation of a **scalar product** by two threads:

Thread 1:

```
for (i=0; i<n/2; i++)
    sum += a[i] * b[i];
```

Thread 2:

```
for (i=n/2+1; i<n; i++)
    sum += a[i] * b[i];
```

Race conditions can arise from the manipulation of variable `sum` possibly leading to errors in the computation.

- **Resolution:** Coordination mechanisms of Pthreads:
  mutex variables, condition variables

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Coordination with Mutex Variables

- Mutex variables are data structures of type `pthread_mutex_t`
  Application: can be used to ensure **mutual exclusion**;

- Mutex variable can be in one of the two states:
  locked and **unlocked**;

- The following procedure can be used to **guarantee mutual exclusion** for a data structure using mutex variables:
  - **Each data structure** A which can be accessed by multiple threads is assigned a **separate mutex variable** M_A.
  - **Before** each (read or write) access to A the accessing thread **locks** the according mutex variable M_A of A.
  - **After** its access to A the corresponding thread **unlocks** the mutex variable M_A.

- **Mutual exclusion** is only guaranteed if **each thread** adheres to this procedure.

Coordination with Mutex Variables

- **Before its first use** the mutex variable has to be **initialized**:

  1st Option: **dynamic** initialization:

```
int pthread_mutex_init(pthread_mutex_t *m,
                       pthread_mutex_attr_t *a);
```

  2nd Option: **static** initialization using a predefined macro:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

- **Locking** the mutex variable `m`:

```
int pthread_mutex_lock(pthread_mutex_t *m);
```

  - If `m` is **free** (unlocked) before the call, then `m` is **locked** by that call.
  - If `m` is **locked** before the call, then the calling thread is **blocked** until the current owner of `m` **unlocks** it.
    $\rightarrow$ The waiting threads are organized in a **queue**;

Coordination with Mutex Variables

- **Unlocking** the mutex variable `m`:

```
int pthread_mutex_unlock(pthread_mutex_t *m);
```

  - If the queue to `m` is **empty**, then `m` has **no owner** after this operation; `m` is afterwards in the **unlocked** (free) state.
  - If the queue to `m` is **not empty**, then **one of the waiting threads** gains control over `m` and is removed from the queue.
    The thread is selected based on **priorities**; the decision is made by a **scheduling algorithm**.

- **Non-blocking** attempt to lock a mutex variable:

```
int pthread_mutex_trylock(pthread_mutex_t *m);
```

  - If the mutex variable `m` is **not locked**, the calling thread gains control over `m`.
  - If `m` is **locked**, the call **immediately** fails (without blocking the calling thread) and returns `EBUSY`.

Example: ensuring mutual exclusion of multiple threads accessing a **global linked list** (1)

Data structures used

```
typedef struct _node_t {
    int index; // Key for sorting
    void *data;
    struct node *next;
} node_t;
typedef struct _list_t {
    node_t *first;
    pthread_mutex_t mutex;
} list_t;
```

Initialization of the list data structure

```
int list_init(list_t *listp) {
    listp->first = NULL;
    listp->mutex = PTHREAD_MUTEX_INITIALIZER;
    return 1;
}
```

Example: **global linked list** (2)

```
void list_insert(int newindex, void *newdata, list_t *listp) {
    node_t *curr, *prev, *new;
    int found = 0;
    pthread_mutex_lock(&listp->mutex);
    {
        for (curr = prev = listp->first; curr != NULL;
             prev = curr, curr = curr->next) {
            if (curr->index == newindex) { found = 1; break; }
            else if (curr->index > newindex) break;
        }
        if (!found) {
            new = (node_t*)malloc(sizeof(node_t));
            new->index = newindex; new->data = newdata;
            new->next = curr;
            if (curr == listp->first) listp->first = new;
            else prev->next = new;
    } }
    pthread_mutex_unlock(&listp->mutex);
}
```

After **initialization** of the list with `list_init()` threads can **add elements** using `list_insert()`; → function `list_insert()` is **thread-safe**.

Danger of deadlocks

- In case of **simultaneous locking** of different mutex variables **deadlocks** can arise:
  - **Example**: Two mutex variables `ma` and `mb` and two threads `T1` and `T2`.
  - Thread T1 first locks `ma` and then `mb`
  - Thread T2 first locks `mb` and then `ma`
  - If thread T1 is interrupted by the scheduler after locking `ma`, thread T2 can lock `mb`, which leads to a **deadlock**.

- **Solution 1**: Use of **locking hierarchy**:
  Each thread locks critical mutex variables in the **same order**.

- **Solution 2**: **back-off strategy** using `pthread_mutex_trylock()`:
  - Each thread locks mutex variables in its individual order using `pthread_mutex_trylock()`.
  - If a **lock attempt fails**, the thread unlocks all mutex variables that it has locked so far (back-off).
  - After back-off the thread starts the entire lock procedure from the beginning.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Condition Variables

- Threads often have to wait for the **fulfillment of a certain condition** before executing an operation.
- **Example**: Producer / consumer threads:
  - Consumer waits on an **empty buffer** for item refill by another thread.
  - Producer waits on a **full buffer** for item removal by another thread.
- **Mutex variable** approach:
  - Each data structure mentioned in the condition is protected by its **own mutex variable**.
  - From time to time the thread evaluates the **condition after locking** the according mutex variables.
  - If the condition is fulfilled, the thread can execute the desired operation; otherwise, the condition has to be evaluated again at a later point in time.

- **Disadvantage** of the approach: Conditions might have to be evaluated very frequently before the thread can execute the desired operation.
  → **active waiting** → unnecessary use of CPU time;

- **Alternative**: **Condition variables** approach (type `pthread_cond_t`) for the realization of a **signaling mechanism**:
  - If the condition is **not fulfilled**, the thread is **blocked** upon condition entry.
  - As soon as the condition is **fulfilled**, the waiting thread is **woken up**.

- **Advantage**: no CPU time use during the waiting time for the fulfillment of the condition.

- Requirements for the implementation of a signaling mechanism:
  - a **condition variable**;
  - a **mutex variable**, which is assigned to the condition variable;
  - a **condition** to be evaluated.
    the evaluation of the condition is protected by the **mutex variable**

Condition variables – Programming

- **Dynamic initialization** of a condition variable:

```
int pthread_cond_init(pthread_cond_t *c,
                      pthread_cond_attr_t *attr);
```

- **Static macro-initialization**:

```
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

- To each condition variable `c` there is a mutex variable `m` that is (implicitly) assigned to `c`.

- Procedure for the waiting for the fulfillment of a condition:
  1. **Lock** the mutex variable `m` assigned to `c`;
  2. Test the **condition** to be evaluated;
  3. If the condition is not fulfilled, the thread gets **blocked on the condition variable**.

- **Blocking** on a condition variable `c`:

```
int pthread_cond_wait(pthread_cond_t *c,
                      pthread_mutex_t *m);
```

  Effect of this call:
  - The calling thread is **blocked** waiting for `c`.
    Unblocking is performed by an **external signal** of another thread.
    Note: without such signal the thread remains **blocked forever**!
  - At the same time, the previous **locks** of `m` (atomic) are **released**;
    $\rightarrow$ another thread can evaluate the condition and send a **wake-up signal** to the waiting thread.

- When the thread blocked on `c` is woken up, it **automatically** tries to **take again control of the associated mutex variable** `m` **again**.

- If this attempt fails, the thread is **blocked** waiting for `m`.
  Since **during the block** on `m` **another thread** can manipulate a variable occurring in the condition, the waiting thread should **re-evaluate** the condition **after re-allocation** of `m`.

Usage of condition variables

- **Typical usage** of condition variables:

```
pthread_mutex_lock(&m);
{
    while (!AnyCondition())
        pthread_cond_wait(&c, &m);
    ...
}
pthread_mutex_unlock(&m);
```

- Mutex variable `m` protects the evaluation of the condition **and** the `pthread_cond_wait()` call.
  $\rightarrow$ The value of the condition **does not change** between the evaluation of the condition and the execution of `pthread_cond_wait` !

- **Requirement: All** threads must lock the mutex variable `m` **before** they manipulate any of the variables occurring in the condition.

Condition variables – signaling operations

- **Signaling** the occurrence of a condition:
  ```
  int pthread_cond_signal(pthread_cond_t *c);
  ```
  Effect of this call:
  - If **no** thread is **blocked** waiting for `c`: **no effect**;
  - If there are threads blocked on `c`, **one** of these threads is woken up, which will then try to gain control of the associated mutex variable `m`.

- Waking up **all** the threads blocked on `c`:
  ```
  int pthread_cond_broadcast(pthread_cond_t *c);
  ```
  - **Each** of the threads woken up tries to gain control of the mutex variable `m` associated to `c`.
  - However, **at most one** of these threads can be successful.
  - All the other threads will be **blocked on** `m`.

- Calls to `pthread_cond_signal()` and `pthread_cond_broadcast()` should only be issued after the thread has evaluated the **associated condition**.
  Evaluation requires the associated mutex variables to be **locked** to ensure consistency.

- Calls to `pthread_cond_signal()` or `pthread_cond_broadcast()` **do not necessarily need** to be protected by a **locking** of `m`.

- A call of these functions in case of a **locked mutex variable** `m` has the following **drawback**: the thread woken up by such a call cannot directly gain control over `m`.

- A call of these functions in case of an **unlocked mutex variable** `m` gives the thread woken up a chance to gain control over `m` **directly**, without blocking on `m`. However, this may lead to **race conditions**.

Condition variables – Example of race conditions

```
typedef struct _my_struct_t {
    pthread_cond_t c;
    pthread_mutex_t m;
    int numThreads;
} my_struct_t;
my_struct_t data = {
    PTHREAD_COND_INITIALIZER, PTHREAD_MUTEX_INITIALIZER, 0 };
```

Assumption: The **main thread** creates a number of threads and sets the variable `numThreads` accordingly. Each thread calls the following function upon **completion**:

```
void SignalCond() {
    pthread_mutex_lock(&data.m);
    data.numThreads--;
    pthread_mutex_unlock(&data.m);
    // At this point the current thread doesn't
    // have a lock on m anymore
    pthread_cond_signal(&data.c);
}
```

A separate control-thread executes the following functions:

```
void WaitForCond() {
    pthread_mutex_lock(&data.m);
    while (data.numThreads > 0)
        pthread_cond_wait(&data.c, &data.m);
    pthread_mutex_unlock(&data.m);
    pthread_cond_destroy(&data.c);
}
```

The program can be executed in the **following order**:

- Thread T_1 executes `SignalCond()`.
  T_1 is **interrupted by the scheduler** after `data.numThreads` is decremented and `m` is freed. Now holds: `data.numThreads == 0`.
- Thread T_2 executes `WaitForCond()`, **does not block** on the condition variable and de-initializes `data.c`.
- T_1 attempts a signaling operation on `data.c`, which has already been de-initialized. This leads to an error!

Solution: In `SignalCond()` move the `pthread_cond_signal()` function call so that it is placed **before** the `pthread_mutex_unlock()` call.

Occurrences of deadlocks with condition variables

- If no other thread is waiting for `c` when `pthread_cond_signal(&c)` is called, the call has **no effect**.
  In particular, the runtime system does **not** report that the signal is set with respect to c. → The signal is **lost**.
- If another thread then calls `pthread_cond_wait(&c)` for this condition variable and **no further signal commands** follows, this thread stays **blocked forever**. → **Deadlock**.

Example (see above):

```
void SignalCond() {
    data.numThreads = 0;
    pthread_cond_signal(&data.c);
}
void WaitForCond() {
    pthread_mutex_lock(&data.m);
    while (data.numThreads > 0)
        pthread_cond_wait(&data.c, &data.m);
    pthread_mutex_unlock(&data.m);
    pthread_cond_destroy(&data.c);
}
```

Deadlock situation

The following **order of execution** is possible for `data.numThreads > 0`:

- Thread T_2 executes `WaitForCond()`, finds `data.numThreads > 0` and is interrupted by the scheduler before calling `pthread_cond_wait()`.
- Thread T_1 executes `SignalCond()`, sets `data.numThreads` to 0 and calls `pthread_cond_signal()`.
  Since T_2 is **not yet waiting on the condition variable**, the calls has **no effect**.
- T_2 calls `pthread_cond_wait()`. T_2 stays blocked forever, since T_1's signal is lost.

Solution: Setting of `data.numThreads` in `SignalCond()` must be protected by locking the `data.m` mutex variable.

Condition variables – further functions

- **Limited-time waiting** for the occurrence of a condition:

```
int pthread_cond_timedwait(pthread_cond_t *c,
                           pthread_mutex_t *m,
                           struct timespec *t);
```

  - The blocking on `c` is **cancelled** as soon as the time given in `t` **has elapsed**.
  - **Return value** in this case: `ETIMEDOUT`.

- Definition of `timespec` (from <time.h>):

```
struct timespec {
    time_t tv_sec;  // seconds
    long   tv_nsec; // nanoseconds
};
```

- Current time request can be performed by a `time()` function call (in <time.h>) or `gettimeofday()` (in <sys/time.h>).
  The value returned is the **absolute time**, not a time interval.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Advanced synchronization mechanisms

- Mutex- and condition variables can be used to implement **complex synchronization mechanisms**.

- **Example**: Reader/Writer locks (**R/W locks**):

  advanced mechanism of access control to an arbitrary data structure:
  - at any point in time **any number of reads** can take place simultaneously;
  - however, only a **single write** can be performed at any time.

- **simple implementation**: define a R/W lock variable as follows:

```
typedef struct _rw_lock_t {
    pthread_mutex_t m;
    pthread_cond_t c;
    int num_r, num_w; // number of currently allowed
                      // read- and write accesses
} rw_lock_t;
```

- **Initialization** of a R/W lock variable:

```
int rw_lock_init(rw_lock_t *rwl) {
    rwl->num_r = rwl->num_w = 0;
    pthread_mutex_init(&rwl->m, NULL);
    pthread_cond_init(&rwl->c, NULL);
}
```

- **Read access request**:
  A **read access** is granted only in case that **no other thread** has a **write permission** (**any number of simultaneous read permissions** is allowed).
  Otherwise, **the requesting thread gets blocked** until the write access in question is ceased.

```
int rw_lock_rlock(rw_lock_t *rwl) {
    pthread_mutex_lock(&rwl->m);
    while (rwl->num_w > 0)
       pthread_cond_wait(&rwl->c, &rwl->m);
    rwl->num_r++;
    pthread_mutex_unlock(&rwl->m);
    return 0; }
```

- **Write access request**:
  A **write access** is granted only in the case that **no other thread** currently possesses a **read or write permission**:

```
int rw_lock_wlock(rw_lock_t *rwl) {
    pthread_mutex_lock(&rwl->m);
    while (rwl->num_w > 0 || rwl->num_r > 0)
       pthread_cond_wait(&rwl->c, &rwl->m);
    rwl->num_w = 1;
    pthread_mutex_unlock(&rwl->m);
    return 0;
}
```

- **Return of the read permission**:

```
int rw_lock_runlock(rw_lock_t *rwl) {
    pthread_mutex_lock(&rwl->m);
    rwl->num_r--;
    if (rwl->num_r == 0)
       pthread_cond_signal(&rwl->c);
    pthread_mutex_unlock(&rwl->m);
    return 0;
}
```

- As soon as the **last read permission** is rerurned, **the waiting write-thread** is woken up.
  note: there can be no other thread waiting for a read permissiom, since a **read-thread** has been executed before.
- **Return of a write permission**: Waking up **of all waiting threads**:

```
int rw_lock_wunlock(rw_lock_t *rwl) {
    pthread_mutex_lock(&rwl->m);
    rwl->num_w = 0;
    pthread_cond_broadcast(&rwl->c);
    pthread_mutex_unlock(&rwl->m);
    return 0;
}
```

- **Observation:** This implementation favors **read permissions over write permissions**: If a thread A was granted **read access** and thread B is waiting for **write access**, **other threads** are granted read access even if they request it **after the write access request** was placed by B;
  $\rightarrow$ Thread B gets his write permission only if **no other thread** requests a **read access** anymore; $\rightarrow$ Danger of **write request starvation**.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Organization of thread cooperation

- **Pipelining model**: Input/Output relation between threads:
  Thread T_i gets the output of thread T_{i-1} as input and produces data for thread T_{i+1}; $\rightarrow$ after the **start-up phase** the threads can work **in parallel**.

- **Master-Slave model** :
  - A **master thread** controls the execution of the program.
  - **Slave threads** perform the necessary calculations.

- **Worker model**: Coordination of program execution and calculations are handled by **equal worker threads**.
  Variation: Calculations to be carried out are stored in **task pools**;

- **Client-Server model**:
  - **Client threads** generate requests.
  - **Server threads** handle these requests by performing the computations requested.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Attribute objects for thread control

- Until now `pthread_create()` has always been called with `NULL` as a default argument. Therefore, **default threads** were created.
- The behavior of threads can be influenced by **attribute objects**.
- **Initialization of an attribute object** with standard attributes:
  ```
  int pthread_attr_init(pthread_attr_t *attr);
  ```
- The Pthread library provides special functions for changing the parameter values;

  Examples:
  - **Release of the return value** of a thread;
  - Setting the **stack size** of a thread;
  - Setting the **start address of the stack** of a thread;
  - **Termination behavior** of threads;
  - **Scheduling** of threads;

Automatic release of a thread's return value

- The attribute **detachstate** influences the **return value of a thread** and is available for all Pthread libraries.

- The **default value** specifies that the return value will be needed **after the thread terminates**, e.g. for `pthread_join()`.
  Effect: The **runtime system** does not free the return value and stores it in an **internal data structure** until some other thread successfully calls `pthread_join()` for this thread.

- If the programmer knows that the return value of a thread **will not be needed** after the termination of that thread, this can be specified when creating the thread:

```
int pthread_attr_getdetachstate(pthread_attr_t *attr,
                                int *detachstate);
int pthread_attr_setdetachstate(pthread_attr_t *attr,
                                int  detachstate);
```

Automatic release of a thread's return value

- Calling `pthread_attr_setdetachstate()` with `PTHREAD_CREATE_DETACHED` as `detachstate` causes the return value to be **freed immediately after termination** of the thread.
  The **default value** is `PTHREAD_CREATE_JOINABLE`.

- The attribute **detachstate** must be specified **before the creation** of the thread, subsequent call has **no effect**.

- An **already created thread** can be **dynamically** set to the detached state by calling:
  ```
  int pthread_detach(pthread_t thread);
  ```
  `EINVAL` is returned if the thread is already detached.

Specification of a thread's stack size

- The attribute **stacksize** can be used if `_POSIX_THREAD_ATTR_STACKSIZE` is defined (in <`unistd.h`>) or when the call `sysconf(_SC_THREAD_ATTR_STACKSIZE)` is successful.

- **Getting or setting the stack size** is done by calling:

```
int pthread_attr_getstacksize(pthread_attr_t *attr,
                              size_t *stacksize);
int pthread_attr_setstacksize(pthread_attr_t *attr,
                              size_t  stacksize);
```

  where `stacksize` is defined as **the size of the stack in bytes**. By calling `pthread_attr_setstacksize` the **minimum size of the local stack** of the thread to be created can be set. The default minimum size depends on the Pthread implementation and is defined by `PTHREAD_STACK_MIN`.

- The **portability of a program** can be **lost** by changing the stack size, as not all Pthread libraries support a change of that attribute.

Specification of the starting address of a thread's stack

- The attribute **stackaddr** can be used if `_POSIX_THREAD_ATTR_STACKADDR` is defined.

- Getting or setting the starting address of the runtime stack is done by:

```
int pthread_attr_getstackaddr(pthread_attr_t *attr,
                              void* *stackaddr);
int pthread_attr_setstackaddr(pthread_attr_t *attr,
                              void*  stackaddr);
```

- Changing this attribute is dangerous!

Example: Creation of a thread with attribute objects

```
pthread_t tid;
pthread_attr_t attr;
size_t stacksize;
...
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
#ifdef _POSIX_THREAD_ATTR_STACKSIZE
    pthread_attr_getstacksize(&attr, &stacksize);
    pthread_attr_setstacksize(&attr,  stacksize * 2);
#endif

pthread_create(&tid, &attr, f, NULL);
...
pthread_exit(NULL);
```

Thread Termination

- Threads can perform **mutual termination** if they detect that the calculations being made by another thread are redundant.

- Threads can call
  `int pthread_cancel(pthread_t tid);`
  to cancel another **thread of the same process** denoted by `tid`. This function call **starts the termination** of the thread `tid`, however, the **return of control** to the calling thread does **not** mean that the thread `tid` is already **terminated**.

- The exact behavior of `pthread_cancel()` depends on the cancellation **state** and **type** of the target thread:
  There are **three cancellation modes** of threads: **no** interruptibility, **delayed** interruptibility and **direct** (asynchronous) **interruptibility**.

Thread Termination – cont.

- Threads can **prevent interruption** from outside by the following function call with `state = PTHREAD_CANCEL_DISABLE`:
  ```
  int pthread_setcancelstate(int state, int *oldstate);
  ```
  The **previous termination state** is saved in `oldstate`.
  The **default value** is `state = PTHREAD_CANCEL_ENABLE`.
  A thread can **re-enable** its **interruptibility** later on by the calling `pthread_setcancelstate()` with `state = PTHREAD_CANCEL_ENABLE`..

- If another thread places a `pthread_cancel()` request and the target thread **can not be interrupted**, then the **cancellation request** is saved and **executed with a delay**.
- In case of `PTHREAD_CANCEL_ENABLE` the target thread is terminated when it reaches a **cancellation point**.

Thread Termination – cancellation points

- **Obligatory cancellation points** are, e.g., calls of:
  - `pthread_cond_wait()`, `pthread_cond_timedwait()`, `pthread_join()`
  - `pthread_testcancel()`
  - `open()`, `read()`, `write()`, `wait()`
- **Optional cancellation points** are calls of:
  `printf()`, `fprintf()`, `scanf()`, `fscanf()`, `fopen()`, `fclose()` etc.
- The `pthread_testcancel()` function call creates an additional **user-defined cancellation point**.
- When a **cancellation point is reached**, the executing thread checks whether **a cancellation request of a different thread** is present. If yes - the thread is **terminated**.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Thread scheduling

- **User threads** defined by the programmer are mapped to kernel threads by the **library scheduler**, which are then in turn allocated to processors by the **operating system scheduler**.

- The programmer can influence the **operation of the library scheduler**, if the macro
  `_POSIX_THREAD_PRIORITY_SCHEDULING`
  is defined in `<unistd.h>`, or if the call of
  `sysconf(_SC_THREAD_PRIORITY_SCHEDULING)`
  is successfull.

- Thread scheduling can be influenced by **scheduling attributes**, which are stored in the data structure

  `struct sched_param` (defined in `<sched.h>`)

  The data structure has at least one entry:

  `int sched_priority;`

  which indicates the **thread's priority**.

Scheduling attributes

- The following aspects can be influenced by scheduling attributes:
  - **Priority** of a thread;
  - **Scheduling method**;
  - **Contention scope**: process or global contention scope;

- The **priority** of a thread is defined by an **integer value**, which must belong to a **predefined range**. The lower and upper bounds of the range can be requested by the following functions:

```
int sched_get_priority_min(int policy);
int sched_get_priority_max(int policy);
```

  where `policy` is the scheduling method used.

Principle of scheduler operation

- **For each priority** the library scheduler maintains **a separate queue** which contains all threads of the same priority that are **ready for execution**.
- If the library scheduler wants to execute a new thread, it picks one from the **non-empty queue with the highest priority**.
- If the queue contains several threads, one of these threads is chosen according to the scheduling method.
- **Effect: Threads with lower priorities might starve**, since threads with the highest priorities are always executed first.

Setting scheduling attributes

- Procedure for **setting a thread's priority**:
  1. **Declaration of a data structure** of the type `sched_param`: sp
  2. **Set** the priority entry: `sp.sched_priority = ...;`
  3. **Pass the priority** to an attribute data structure `attr` with the help of the function:

```
int pthread_attr_setschedparam(pthread_attr_t *attr,
                               struct sched_param *sp);
```

  4. Creation of a thread with the corresponding attribute data structure: `pthread_create(&tid, &attr, ...);`

- **Request** of the **priority** stored in an attribute structure is done by calling:

```
int pthread_attr_getschedparam(pthread_attr_t *attr,
                               struct sched_param *sp);
```

Priorities and scheduling methods

- The Pthread standard defines that **at least 32 priority levels** have to be available.
- **Scheduling methods** define the **execution behavior of threads belonging to the same priority level**. Three scheduling methods are supported:

  1. `SCHED_FIFO`: First-In-First-Out scheduling;
  2. `SCHED_RR`: Round-Robin scheduling;
  3. `SCHED_OTHER`: implementation-specific scheduling;

- The Pthread standard **does not specify a default scheduling method**.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

One-time initialization

- In some situations, a certain task (e.g. data structure initialization) **has to be executed exactly once**. However, it is not always clear upon program start **which thread** should execute that task.

- **The simplest solution:** Use a **boolean variable** protected by a mutex variable **initialized** to FALSE. The task to be performed is executed only if the boolean variable is FALSE. The first thread arriving at the critical task, sets the value of the variable to TRUE and executes the task.

One-time initialization with pthread_once()

- **Another solution:** Use of an additional Pthreads mechanism:
  - **Declaration** of a control variable of type `pthread_once_t`.
  - **Initialization** of that control variable with the macro `PTHREAD_ONCE_INIT`.
  - The task to be executed only once is put into a **separate function without parameters** `once_routine`.
  - To perform the **one-time execution** of the task, the following function is called (possibly multiple times by different threads):

```
int pthread_once(pthread_once_t *control,
                 void (*once_routine)(void));
```

The **following effect** is achieved:

- Only the **first call** leads to the execution of `once_routine()`.
- If another thread is already executing `once_routine()`, then control is **only returned** when the execution of `once_routine()` is **finished**.
- If `once_routine()` is already **completely** finished, the control is immediately returned to the calling thread.

Section outline

1. Pthread Programming
   - Creation and Merging Threads
   - Thread Coordination with Pthreads
   - Coordination with Mutex Variables
   - Condition Variables
   - Extended Lock Mechanism
   - Organization of thread cooperation
   - Thread Attributes and Cancellation
   - Thread scheduling with Pthreads
   - Thread and process management
   - One-time initialization
   - Thread-specific data

Thread-specific data

- Each thread of the process can access all **globally** and **dynamically allocated** variables. **Local variables** of a thread are stored in its own runtime stack; their lifetime depends on the function in which they are declared.
- The creation of variables with a **lifetime** spanning the whole runtime of a thread can be performed as follows:
  - Use of **global variables**. **Disadvantage**: All threads have access.
  - Passing as a parameter to each called function. **Disadvantage**: Parameters have to be passed to all function calls.
  - **Thread-specific data** as an additional mechanism: Use **process-global keys** to which each thread can associate its data.
- **Creation of a key** of type `pthread_key_t` by a function call of

```
int pthread_key_create(pthread_key_t *key,
                       void (*destructor)(void*));
```

  where `destructor()` is a **deallocation function**, which is called upon thread **termination**.
  Each key can be **initialized only once**! This can be achieved by using **one-time initialization**.

Thread-specific data – cont.

- **Association of value** to a key by calling:

```
int pthread_setspecific(pthread_key_t key,
                        void *value);
```

  The value saved is a pointer to the data, not the data itself. Only the calling thread can later access that data!

- **Data associated** with a key can be retrieved by calling the function

```
void* pthread_getspecific(pthread_key_t key);
```

  Each thread gets the value that **it** has previously stored.

- **Removal of a key** is done by

```
int pthread_key_delete(pthread_key_t key);
```

  Any stored data is lost.